

AutoJudge: Programming Problem Difficulty Prediction

GOLLA SAHITHI,23125012

1. Introduction

Programming platforms host problems with varying levels of difficulty, and assigning difficulty labels manually can be subjective and time-consuming. The objective of this project, **AutoJudge**, is to automatically predict the difficulty of programming problems using machine learning.

Given a problem's textual description, the system predicts both a **difficulty class** (Easy, Medium, or Hard) and a **numerical difficulty score**. The project demonstrates a complete machine learning pipeline from data preprocessing to deployment through a web interface.

2. Dataset Description

The dataset used consists of programming problem descriptions stored in JSONL format. Each problem includes textual fields such as the problem statement, input and output descriptions, and difficulty annotations.

The dataset contains **4112 programming problems**, with difficulty scores ranging from approximately **1 to 10**, making it suitable for both classification and regression tasks.

3. Data Preprocessing

The dataset was converted into a structured format for analysis. Missing values were handled, and nested input/output fields were flattened into plain text.

All relevant textual fields were combined into a single text feature. The combined text was cleaned by converting it to lowercase, removing unnecessary special characters, and normalizing whitespace. These steps helped reduce noise and improve model performance.

4. Feature Engineering

Textual data was transformed into numerical features using **TF-IDF vectorization**. In addition, custom features such as **text length** and **mathematical symbol count** were extracted to capture problem complexity.

The custom features were scaled using **StandardScaler**, and all features were combined to form the final input to the models.

5. Models and Evaluation

Classification

For classification, **Logistic Regression** was used as a baseline model, followed by a **Support Vector Machine (SVM)** as the final classifier. Model performance was evaluated using accuracy and a confusion matrix.

Regression

For regression, a **Gradient Boosting Regressor** was used to predict the difficulty score. The model achieved:

- **MAE:** 1.69
- **RMSE:** 2.03

These results indicate acceptable predictive performance given the difficulty score range.

6. Web Interface

A **Streamlit-based web application** was developed to provide an interactive interface. Users can input problem descriptions and receive predicted difficulty class and score. The application runs locally and loads pre-trained models for efficient prediction.

7. Conclusion

This project demonstrates how classical machine learning techniques can be applied to automatically assess programming problem difficulty using textual information. AutoJudge successfully integrates preprocessing, feature extraction, model training, and deployment into a single system. Future improvements may include incorporating additional features or expanding the dataset.

Paste the programming problem details below:

Problem Description

You are given a string S consisting only of the characters '(' and ')'.
A balanced substring is one where every opening parenthesis '(' has a matching closing parenthesis ')' in the correct order.

Input Description

A single line containing the string S ($1 \leq |S| \leq 10^5$).

Output Description

A single integer — the length of the longest balanced substring.

Predict

 **Prediction Results**

Predicted Difficulty Class: Hard

Predicted Difficulty Score: 3.30